

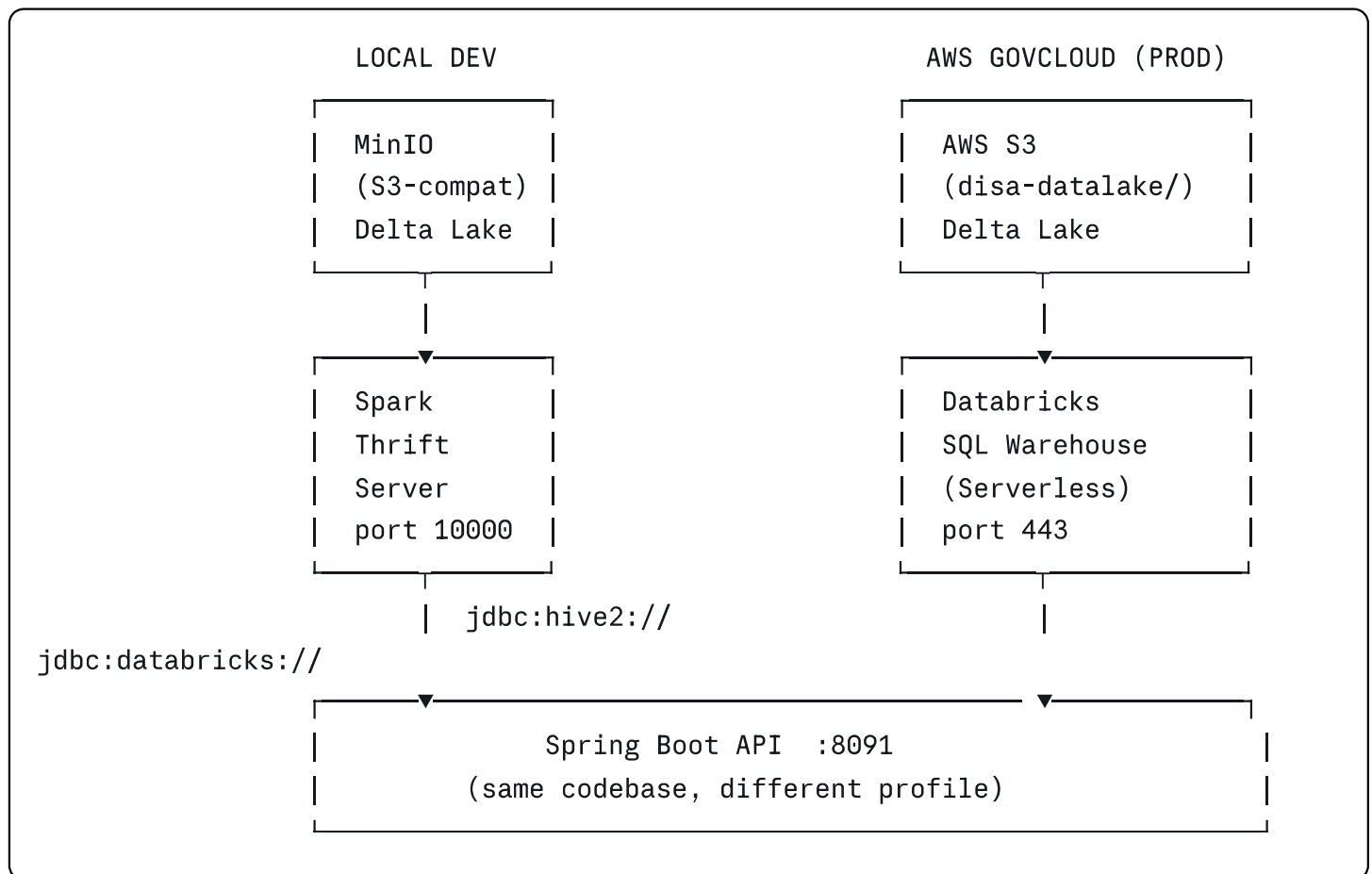
OE Data Intelligence Platform — Developer Architecture Guide

Why Are We Using Hive? We're Not a Hadoop Shop.

Great question. Short answer: **we're not really using Hive — we're using its wire protocol.**

Here's what's actually happening and why.

The Big Picture



The Spring Boot application **does not change between environments**. Only one line in `application-thrift.yml` vs `application.yml` differs — the JDBC URL.

What Is the Spark Thrift Server?

Apache Spark ships with a built-in component called the **HiveServer2-compatible Thrift Server**. Its job is simple: accept SQL queries over a network port and execute them on Spark's distributed query engine.

Think of it as Spark wearing a Hive costume for the network layer.



Why does this matter for us? Databricks SQL Warehouse uses the exact same HiveServer2 wire protocol on port 443. By pointing our local environment at a Spark Thrift Server on port 10000, we can develop and test SQL queries locally that will run **identically** on Databricks in production. No mocking. No in-memory H2 hacks. Real Delta Lake. Real Spark SQL.

Why Is the Hive JDBC Driver in Our pom.xml?

JDBC is the standard Java database connectivity API. To talk to anything over JDBC you need a **driver** — a JAR that knows the wire protocol of that database.

Database	Driver JAR
PostgreSQL	postgresql-42.x.jar
MySQL	mysql-connector-j.jar
Oracle	ojdbc.jar
Hive/Spark/Databricks	hive-jdbc-standalone.jar

All three — local Spark Thrift Server, Databricks Community Edition, Databricks SQL Warehouse on AWS GovCloud — use the HiveServer2 protocol. So one driver covers all environments.

The Databricks JDBC driver (`databricks-jdbc-2.6.38.jar`) is also in the project. It's a more modern, JDK-21-native driver for production use. We use Hive for local/dev because it's open source and works with any HiveServer2-compatible endpoint.

Why Is the Hive JAR Patched and Checked Into git?

The official `hive-jdbc-3.1.3-standalone.jar` on Maven Central bundles several libraries inside it that conflict with Spring Boot 4:

Bundled class (inside Hive JAR)	Conflict with Spring Boot 4
<code>org/slf4j/</code> — SLF4J 1.x classes	Spring Boot 4 uses SLF4J 2.x + Logback
<code>org/apache/jasper/</code> — JSP engine	Spring Boot 4 uses Jakarta EE 10 servlet API

These are physically embedded class files inside the fat JAR — not separate Maven artifacts. Maven `<exclusion>` tags cannot remove them. The only fix is to open the JAR and delete those directories, then re-package it.

We did that once and committed the patched JAR to `libs/hive-jdbc-3.1.3-standalone.jar`. This is intentional. Any developer who clones the repo gets a working build with zero manual steps. The pom.xml uses `<scope>system</scope>` to reference this local file directly instead of downloading from Maven Central.

What Is Delta Lake and Why Does It Matter?

Delta Lake is an open-source storage layer that sits on top of regular files (Parquet format) stored in S3/MinIO. It adds three things that plain Parquet doesn't have:

- **ACID transactions** — consistent reads even while data is being written
- **Schema enforcement** — rejects data that doesn't match the table definition
- **Time travel** — query the table as it existed at any past point in time

Our pipeline writes Gold tables as Delta format to MinIO. The Spark Thrift Server reads those Delta files and serves them as SQL tables. Spring Boot queries those tables over JDBC as if they were a regular database.



What Is the Hive Metastore?

The Hive metastore is a catalog — a registry that maps **table names** to **file paths**.

When you query `SELECT * FROM gold.device_health_summary`, Spark needs to know:

- Where are the actual files? (`s3a://gold/device_health_summary/`)
- What is the schema? (columns, data types)
- What format are they? (Delta Lake, Parquet, CSV...)

That mapping lives in the **metastore**. In production Databricks, this is Unity Catalog. In our local Docker stack, it's an embedded Derby database inside the Thrift Server container.

This is why we run the Beeline command after startup:

```
cmd

docker exec oe-spark-thrift /opt/spark/bin/beeline -u "jdbc:hive2://localhost:10000/"
-e "CREATE TABLE IF NOT EXISTS gold.app_health_summary
    USING DELTA LOCATION 's3a://gold/app_health_summary';"
```

The Delta files exist in MinIO already (written by the pipeline). But Spark's metastore doesn't know about them until we register them. `CREATE TABLE ... USING DELTA LOCATION` is just "hey Spark, when someone queries `gold.app_health_summary`, look here."

Important: The Derby metastore is ephemeral — it resets when the Thrift Server container restarts. Re-run the Beeline registration after every `docker compose restart`. In production Databricks, Unity Catalog persists permanently.

Why Does the `HiveTransactionConfig` Class Exist?

Spring's `@Transactional` annotation is on every service method. Normally it works like this:

1. BEGIN transaction (`connection.setAutoCommit(false)`)
2. Execute SQL query
3. COMMIT transaction (`connection.commit()`)
4. If error → ROLLBACK (`connection.rollback()`)

Hive/Spark Thrift Server is a **read-only analytics engine**. It doesn't support database transactions. Calling `commit()` or `rollback()` on a Hive connection throws:

```
SQLFeatureNotSupportedException: Method not supported
```

We can't remove `@Transactional` from the service classes because on AWS Databricks those annotations work fine (Databricks JDBC is fully transaction-capable). Instead, `HiveTransactionConfig` replaces the transaction manager with a version that silently skips `commit()` and `rollback()` calls — only when the `thrift` profile is active.

```
java

@Configuration
@Profile("thrift") // ← only active in local dev
public class HiveTransactionConfig {
    @Bean @Primary
    public DataSourceTransactionManager transactionManager(DataSource ds) {
        return new DataSourceTransactionManager(ds) {
            @Override protected void doCommit(...) { /* no-op */ }
            @Override protected void doRollback(...) { /* no-op */ }
        };
    }
}
```

On AWS GovCloud, this class is inactive. The standard transaction manager handles everything normally.

The Three Profiles Explained

Profile	Used When	Database	Where Data Lives
<code>local</code> / <code>mock</code>	Unit testing, offline	H2 in-memory	Nowhere — fabricated
<code>thrift</code>	Local Docker development	Spark Thrift Server	MinIO (local S3)
<code>(default)</code>	AWS GovCloud production	Databricks SQL Warehouse	AWS S3 GovCloud

Switch profiles by changing one Maven argument:

```
bash

mvn spring-boot:run -Dspring-boot.run.profiles=thrift    # local Docker
mvn spring-boot:run -Dspring-boot.run.profiles=mock      # no Docker needed
mvn spring-boot:run                                     # production Databricks
```

Developer Quick Reference

First-Time Setup (after git clone)

```
bash

cd oe-ingestion-pipeline
docker compose up -d
# Wait ~2 min for Spark Thrift Server to start
```

Ingest Data

```
bash

# In JupyterLab terminal (http://localhost:8888, token: databricks)
python scripts/generate_mock_logs.py --days 3
python run_pipeline.py
```

Register Tables in Thrift Server (required after every restart)

```
cmd
```

```
docker exec oe-spark-thrift /opt/spark/bin/beeline -u "jdbc:hive2://localhost:10000/
```

Start Spring Boot API

```
bash

cd oe-backend-complete
mvn spring-boot:run -Dspring-boot.run.profiles=thrift
```

Test Endpoints

```
bash

curl http://localhost:8091/actuator/health
curl http://localhost:8091/api/app-health
curl http://localhost:8091/api/device-health
curl http://localhost:8091/api/data-sources
curl http://localhost:8091/api/network-performance
```

Key URLs

Service	URL	Credentials
JupyterLab	http://localhost:8888	token: databricks
MinIO Console	http://localhost:9001	minioadmin / minioadmin
Spark Master UI	http://localhost:8080	none
Spring Boot API	http://localhost:8091	none
Spark UI (Thrift)	http://localhost:4041	none